Kernel: SageMath 10.7

In [2]:

```
from itertools import combinations

# ============================================================
# Bases
# ============================================================

# Basis of Λ³(R⁶) (20 elements)
basis3 = list(combinations(range(6), 3))

# Basis of Λ²(R⁶) (15 elements)
basis2 = list(combinations(range(6), 2))

# Fast lookup: pair -> row index
basis2_pos = {pair: m for m, pair in enumerate(basis2)}

EPS = 1e-12


# ============================================================
# Build α from 20 coefficients
# ============================================================

def alpha_from_coeffs(coeffs):
    """
    Input:
        coeffs = list of 20 numbers

    Output:
        dictionary representing α ∈ Λ³(R⁶)
    """
    assert len(coeffs) == 20, f"Need 20 coefficients, got {len(coeffs)}"

    return {
        basis3[m]: RR(coeffs[m])
        for m in range(20)
        if abs(coeffs[m]) > EPS
    }


# ============================================================
# Build α from decomposable terms
# ============================================================

def alpha_from_wedge(*triples):
    """
    Each argument is (u,v,w)

    u,v,w are vectors in R^6.

    Returns:
        α = Σ (u∧v∧w)
    """

    alpha = {idx: RR(0) for idx in basis3}
```

```
    for (u, v, w) in triples:

        for (i, j, k) in basis3:

            det = (
                u[i]*(v[j]*w[k] - v[k]*w[j])
                - u[j]*(v[i]*w[k] - v[k]*w[i])
                + u[k]*(v[i]*w[j] - v[j]*w[i])
            )

            alpha[(i, j, k)] += det

    return {
        k: v
        for k, v in alpha.items()
        if abs(v) > EPS
    }


# ============================================================
# Interactive input
# ============================================================

def alpha_from_input():

    print("\nBasis order:\n")

    for m, idx in enumerate(basis3):
        name = "e" + "".join(str(i + 1) for i in idx)
        print(f"{m+1:2d}: {name}")

    print("\nEnter 20 coefficients separated by spaces:\n")

    coeffs = list(map(float, input().split()))

    return alpha_from_coeffs(coeffs)


# ============================================================
# Contraction i_{e_s}(α)
# ============================================================

def contraction(alpha, s):

    result = [RR(0)] * 15

    for (i, j, k), coeff in alpha.items():

        if abs(coeff) < EPS:
            continue

        if s == i:
            result[basis2_pos[(j, k)]] += coeff

        elif s == j:
            result[basis2_pos[(i, k)]] -= coeff
```

```
        elif s == k:
            result[basis2_pos[(i, j)]] += coeff

    return result


# ============================================================
# Build ϕ_α : R⁶ → Λ²(R⁶)
#
# Returns the 15×6 matrix
# ============================================================

def phi_matrix(alpha):

    cols = [contraction(alpha, s) for s in range(6)]

    return matrix(RR, list(map(list, zip(*cols))))


# ============================================================
# Rank of ϕ_α
# ============================================================

def phi_rank(alpha):

    M = phi_matrix(alpha)

    return M.rank()


# ============================================================
# Analysis
# ============================================================

def analyze(alpha):

    print("\nNonzero coordinates of α:\n")

    for idx, coeff in sorted(alpha.items()):

        name = "e" + "".join(str(i + 1) for i in idx)

        print(f"{name} : {coeff}")

    M = phi_matrix(alpha)

    print("\nϕ_α =\n")
    print(M)

    r = M.rank()

    print("\nrank(ϕ_α) =", r)

    return r
```

```
# ============================================================
# Generic basis builder
# ============================================================

def basis_k(k):
    return list(combinations(range(6), k))


# ============================================================
# Sign of permutation
# ============================================================

def permutation_sign(seq):
    """
    Returns sign of permutation needed
    to sort seq into increasing order.
    """

    seq = list(seq)

    sign = 1

    for i in range(len(seq)):
        for j in range(i + 1, len(seq)):
            if seq[i] > seq[j]:
                sign *= -1

    return sign


# ============================================================
# Wedge product
# ============================================================

def wedge(A, B):
    """
    A,B represented as dictionaries

    Example:

        {(0,1):2}
        means 2 e12

    Returns A∧B.
    """

    result = {}

    for I, a in A.items():

        for J, b in B.items():

            K = I + J

            # repeated index => zero
```

```
            if len(set(K)) < len(K):
                continue

            sign = permutation_sign(K)

            Ksorted = tuple(sorted(K))

            result[Ksorted] = (
                result.get(Ksorted, RR(0))
                + sign*a*b
            )

    return {
        k: v
        for k, v in result.items()
        if abs(v) > EPS
    }


def is_degenerate(alpha):

    M = phi_matrix(alpha)

    r = M.rank()

    return r < 6


# ============================================================
# Basis of Λ⁴(R⁶)
# ============================================================

basis4 = list(combinations(range(6), 4))

basis4_pos = {
    idx: m
    for m, idx in enumerate(basis4)
}
def permutation_sign(seq):

    seq = list(seq)
    s = 1

    for i in range(len(seq)):
        for j in range(i+1, len(seq)):
            if seq[i] > seq[j]:
                s *= -1

    return s

# ============================================================
# Matrix of η ↦ ηΛω
#
# 15 × 6 matrix
# ============================================================

def wedge1_with_alpha_matrix(alpha):
```

```
    M = matrix(RR, 15, 6)

    for s in range(6):

        for triple, coeff in alpha.items():

            if s in triple:
                continue

            quad = tuple(sorted((s,) + triple))

            sign = permutation_sign((s,) + triple)

            row = basis4_pos[quad]

            M[row, s] += sign * coeff

    return M

# ============================================================
# Find α such that α∧ω = 0
# ============================================================

def linear_factor_candidates(omega):

    M = wedge1_with_alpha_matrix(omega)

    K = M.right_kernel()

    return K


def solve_beta(alpha_vec, omega):

    A = matrix(RR, 20, 15)

    for col, (i,j) in enumerate(basis2):

        for s in range(6):

            a = alpha_vec[s]

            if abs(a) < EPS:
                continue

            if s in (i,j):
                continue

            quad = tuple(sorted((s,i,j)))

            sign = permutation_sign((s,i,j))

            row = basis3.index(quad)

            A[row,col] += sign*a
```

```
    rhs = vector(
        RR,
        [omega.get(idx,0) for idx in basis3]
    )

    sol = A.solve_right(rhs)

    return sol

# ============================================================
# Rank of β
# ============================================================

def beta_rank(beta_vec):

    B = matrix(RR, 6, 6)

    for coeff, (i,j) in zip(beta_vec, basis2):

        if abs(coeff) < EPS:
            continue

        B[i,j] = coeff
        B[j,i] = -coeff

    return B.rank()



# ============================================================
# Degenerate classification
# ============================================================

def classify_degenerate(omega):

    K = linear_factor_candidates(omega)

    if K.dimension() == 0:
        raise ValueError("Form is not degenerate")

    alpha_vec = K.basis()[0]

    beta_vec = solve_beta(alpha_vec, omega)

    r = beta_rank(beta_vec)

    if r == 0:

        orbit = "Type (6)"
        normal_form = "0"
        real_rank = 0

    elif r == 2:

        orbit = "Type (5)"
        normal_form = "e123"
```

```
        real_rank = 1

    elif r == 4:

        orbit = "Type (4)"
        normal_form = "e125 + e345"
        real_rank = 2

    else:

        orbit = "Unknown"
        normal_form = None
        real_rank = None

    return {
        "orbit": orbit,
        "normal_form": normal_form,
        "real_rank": real_rank,
        "alpha": alpha_vec,
        "beta": beta_vec,
        "beta_rank": r
    }


# ============================================================
# First stage of Bryant classification
# ============================================================

def classify(omega):

    if is_degenerate(omega):

        return classify_degenerate(omega)

    else:

        return {
            "orbit": "Nondegenerate",
            "next_step": "Compute Hitchin lambda"
        }



# ============================================================
# Basis of Λ⁵(R⁶)
# ============================================================

basis5 = list(combinations(range(6),5))

basis5_pos = {
    idx:m
    for m,idx in enumerate(basis5)
}


def contraction_form(alpha,s):
    """
```

```
    returns i_{e_s}(alpha)
    as a dictionary
    """

    result = {}

    for (i,j,k), coeff in alpha.items():

        if s == i:
            result[(j,k)] = result.get((j,k),RR(0)) + coeff

        elif s == j:
            result[(i,k)] = result.get((i,k),RR(0)) - coeff

        elif s == k:
            result[(i,j)] = result.get((i,j),RR(0)) + coeff

    return result


def beta_s(alpha,s):
    """
    β_s=(i_{e_s}α)∧α

    returns a 5-form
    """

    return wedge(
        contraction_form(alpha,s),
        alpha
    )



# ============================================================
# Missing index of a basis 5-form
#
# Example:
#   (1,2,3,4,5) -> 0
#   (0,2,3,4,5) -> 1
#
# Since every basis element of Λ⁵(R⁶) misses exactly one index.
# ============================================================

def missing_index(five_tuple):

    for i in range(6):
        if i not in five_tuple:
            return i


# ============================================================
# Convert a 5-form into a vector in R⁶
#
# This implements Hitchin's map
#
#     A : Λ⁵(R⁶) → (R⁶)*
```

```
#
# using the volume form
#
#     vol = e123456.
#
# Example:
#
#     e23456 ↦ e¹
#     e13456 ↦ -e²
#
# The sign is determined by:
#
#     e^m ∧ beta = ± vol
# ============================================================

def fiveform_to_vector(beta):

    v = vector(RR, [0]*6)

    for idx, coeff in beta.items():

        # missing basis index
        m = missing_index(idx)

        # sign of
        # (m, i1, i2, i3, i4, i5)
        sign = permutation_sign((m,) + idx)

        v[m] += sign * coeff

    return v


# ============================================================
# Hitchin operator
#
# K_α : R⁶ → R⁶
#
# defined by
#
#     K_α(e_s)
#       = A( (i_{e_s} α) ∧ α )
#
# Columns are K_α(e₁),...,K_α(e₆).
# ============================================================

def hitchin_operator(alpha):

    cols = []

    for s in range(6):

        # β_s = (i_{e_s} α) ∧ α
        beta = beta_s(alpha, s)

        # convert 5-form to vector
        col = fiveform_to_vector(beta)
```

```
        cols.append(col)

    # columns -> matrix
    K = matrix(RR, list(map(list, zip(*cols))))

    return K


# ============================================================
# Hitchin quartic invariant
#
# λ(α) = (1/6) tr(K_α²)
#
# For stable forms:
#
#     K_α² = λ(α) I
#
# ============================================================

def hitchin_lambda(alpha):

    K = hitchin_operator(alpha)

    return (K*K).trace()/6


# ============================================================
# Stable orbit classification
#
# λ > 0  : positive stable orbit
# λ < 0  : negative stable orbit
# λ = 0  : degenerate orbit
# ============================================================

def stable_type(alpha):

    lam = hitchin_lambda(alpha)

    if lam > EPS:
        return "positive stable"

    elif lam < -EPS:
        return "negative stable"

    else:
        return "degenerate"


# ============================================================
# Convenience function
# ============================================================

def analyze_hitchin(alpha):

    K = hitchin_operator(alpha)
```

```
    print("\nK_alpha =\n")
    print(K)

    print("\nK_alpha^2 =\n")
    print(K*K)

    lam = hitchin_lambda(alpha)

    print("\nlambda(alpha) =", lam)

    print("\nOrbit type =", stable_type(alpha))

    return K, lam


# ============================================================
# Λ³(R⁶) dictionary <-> coordinate vector
# ============================================================

def alpha_to_vector(alpha):

    v = vector(RR, [0]*20)

    for idx, coeff in alpha.items():
        v[basis3.index(idx)] = coeff

    return v


def vector_to_alpha(v):

    alpha = {}

    for m, coeff in enumerate(v):
        if abs(coeff) > EPS:
            alpha[basis3[m]] = RR(coeff)

    return alpha

# ============================================================
# Dual action on basis 1-forms
# ============================================================

def dual_basis_form(A, j):

    form = {}

    for i in range(6):

        c = A[j,i]

        if abs(c) > EPS:
            form[(i,)] = c

    return form

# ============================================================
```

```
# Exterior power Λ³(A)
# ============================================================

def exterior_power_3(A):

    M = matrix(RR, 20, 20)

    for col, I in enumerate(basis3):

        i,j,k = I

        ei = dual_basis_form(A, i)
        ej = dual_basis_form(A, j)
        ek = dual_basis_form(A, k)

        image = wedge(wedge(ei,ej),ek)

        for J, coeff in image.items():

            row = basis3.index(J)

            M[row,col] += coeff

    return M


# ============================================================
# Hitchin quantity K*Ω
# ============================================================

def hitchin_star_alpha(alpha):

    K = hitchin_operator(alpha)

    Lambda3K = exterior_power_3(K)

    v = alpha_to_vector(alpha)

    image = Lambda3K * v

    return vector_to_alpha(image)


# ============================================================
# Hitchin decomposition
# ============================================================

def hitchin_decomposition(alpha):

    lam = hitchin_lambda(alpha)

    if lam <= EPS:
        raise ValueError("Need positive stable form")

    mu3 = lam^(3/2)

    Kstar_alpha = hitchin_star_alpha(alpha)
```

```
    v1 = alpha_to_vector(alpha)
    v2 = alpha_to_vector(Kstar_alpha)

    alpha_part = (mu3*v1 + v2)/(2*mu3)
    beta_part  = (mu3*v1 - v2)/(2*mu3)

    return (
        vector_to_alpha(alpha_part),
        vector_to_alpha(beta_part)
    )



# def hitchin_star_alpha_transpose(alpha):

#     K = hitchin_operator(alpha)

#     Lambda3K = exterior_power_3(K.transpose())

#     v = alpha_to_vector(alpha)

#     image = Lambda3K * v

#     return vector_to_alpha(image)

# lam = hitchin_lambda(omega)
# mu3 = lam^(3/2)

# Kstar = hitchin_star_alpha_transpose(omega)

# alpha2 = vector_to_alpha(
#     (mu3*alpha_to_vector(omega)
#      + alpha_to_vector(Kstar))/(2*mu3)
# )

# beta2 = vector_to_alpha(
#     (mu3*alpha_to_vector(omega)
#      - alpha_to_vector(Kstar))/(2*mu3)
# )

def hitchin_decomposition_negative(omega):

    lam = hitchin_lambda(omega)

    if lam >= -EPS:
        raise ValueError("Need lambda < 0")

    mu = sqrt(-lam)
    mu3 = mu^3

    KstarOmega = hitchin_star_alpha(omega)

    vOmega = alpha_to_vector(omega)
    vK     = alpha_to_vector(KstarOmega)

    alpha_complex = (
```

```
        vOmega + (I/mu3)*vK
    ) / 2

    alpha_bar = (
        vOmega - (I/mu3)*vK
    ) / 2

    return alpha_complex, alpha_bar

# ============================================================
# Nondegenerate classification
# ============================================================

def classify_nondegenerate(alpha):

    lam = hitchin_lambda(alpha)

    if lam > EPS:

        alpha_part, beta_part = hitchin_decomposition(alpha)

        return {
            "orbit" : "Type (1)",
            "lambda": lam,
            "alpha" : alpha_part,
            "beta"  : beta_part
        }

    elif lam < -EPS:

        alpha_complex, alpha_bar = \
            hitchin_decomposition_negative(alpha)

        return {
            "orbit" : "Type (2)",
            "lambda": lam,
            "alpha" : alpha_complex,
            "alpha_bar" : alpha_bar
        }

    else:

        return {
            "orbit" : "Type (3)",
            "lambda": 0
        }


def classify_full(alpha):

    if is_degenerate(alpha):

        return classify_degenerate(alpha)

    return classify_nondegenerate(alpha)
```

```
degenerate_test_cases = [

    # 1. Type 5: Decomposable product of (e0 + e1), (e2 + e3), (e4 + e5)
    {
        (0, 2, 4): 1, (0, 2, 5): 1, (0, 3, 4): 1, (0, 3, 5): 1,
        (1, 2, 4): 1, (1, 2, 5): 1, (1, 3, 4): 1, (1, 3, 5): 1
    },

    # 2. Type 4: Factorable into 1-form (e4 - e5) wedged with rank-4 2-form
(e0∧e1 + e2∧e3)
    {
        (0, 1, 4): 1, (0, 1, 5): -1,
        (2, 3, 4): 1, (2, 3, 5): -1
    },

    # 3. Type 6: Empty dictionary representing the clean zero form
    {},

    # 4. Type 4: Highly scrambled using all 6 dimensions with a common factor of
(e0 + e1 + e2 + e3 + e4 + e5)
    {
        (0, 1, 2): 1, (0, 1, 3): 1, (0, 1, 4): 1, (0, 1, 5): 1,
        (0, 2, 3): 1, (1, 2, 3): 1, (2, 3, 4): 1, (2, 3, 5): 1
    },

    # 5. Type 5: Decomposable product of e0 ∧ e1 ∧ (e2 + e3 + e4 + e5)
    {
        (0, 1, 2): 1, (0, 1, 3): 1, (0, 1, 4): 1, (0, 1, 5): 1
    },

    # 6. Type 6: Explicit zero entries (edge case to test if your code drops
zero weights)
    {
        (0, 1, 2): 0, (3, 4, 5): 0
    },

    # 7. Type 4: Symplectic sign-split variation from equation A.37: (e0∧e1 -
e3∧e4) ∧ (e2 + e5)
    {
        (0, 1, 2): 1, (0, 1, 5): 1,
        (2, 3, 4): -1, (3, 4, 5): -1
    },

    # 8. Type 5: Decomposable product of (e0 + e5) ∧ (e1 + e4) ∧ e2
    {
        (0, 1, 2): 1, (0, 2, 4): 1,
        (1, 2, 5): -1, (2, 4, 5): -1
    },

    # 9. Type 4: Factorable into e0 ∧ (e1∧e2 + e3∧e4 + e3∧e5) -> Rank 4 block
mixed with e3
    {
        (0, 1, 2): 2, (0, 3, 4): 2, (0, 3, 5): 2
    },

    # 10. Type 6: Another standard empty structure representing the zero form
    {},
```

```
    # 11. Type 5: Decomposable product of (e0 + e1) ∧ (e1 + e2) ∧ (e2 + e3)
    {
        (0, 1, 2): 1, (0, 1, 3): 1,
        (0, 2, 3): 1, (1, 2, 3): 1
    },

    # 12. Type 4: Linear factor is e3, wedged with rank-4 2-form (e0∧e1 + e2∧e4
- e1∧e5)
    {
        (0, 1, 3): 1, (2, 3, 4): 1, (1, 3, 5): -1
    },

    # 13. Type 5: Decomposable product of (e0 - e1) ∧ (e2 - e3) ∧ (e4 - e5)
    {
        (0, 2, 4): 1, (0, 2, 5): -1, (0, 3, 4): -1, (0, 3, 5): 1,
        (1, 2, 4): -1, (1, 2, 5): 1, (1, 3, 4): 1, (1, 3, 5): -1
    },

    # 14. Type 4: Factorable into (e0 + e2 + e4) ∧ (e1∧e3 + e1∧e5) -> Rank 4
distribution
    {
        (0, 1, 3): 1, (0, 1, 5): 1,
        (1, 2, 3): 1, (1, 2, 5): 1,
        (1, 3, 4): 1, (1, 4, 5): 1
    },

    # 15. Type 6: Evaluating to zero with mixed cancelled elements
    {},

    # 16. Type 5: Decomposable product of (e0 + e1 + e2) ∧ e3 ∧ e4
    {
        (0, 3, 4): 1, (1, 3, 4): 1, (2, 3, 4): 1
    },

    # 17. Type 4: Linear factor (e0 + e1), wedged with rank-4 symmetric shift
(e2∧e3 + e4∧e5)
    {
        (0, 2, 3): 1, (0, 4, 5): 1,
        (1, 2, 3): 1, (1, 4, 5): 1
    },

    # 18. Type 6: Empty evaluation case
    {},

    # 19. Type 5: Decomposable product of (e0 + e3) ∧ (e1 + e4) ∧ (e2 + e5)
    {
        (0, 1, 2): 1, (0, 1, 5): 1, (0, 2, 4): -1, (0, 4, 5): 1,
        (1, 2, 3): 1, (1, 3, 5): -1, (2, 3, 4): 1, (3, 4, 5): 1
    },

    # 20. Type 4: Linear factor e5, wedged with rank-4 matrix (e0∧e1 + 3*e2∧e3)
    {
        (0, 1, 5): 1, (2, 3, 5): 3
    }
]
```

```
for i in degenerate_test_cases:
    print(classify_degenerate(i))
    print("="*60)
```

Out[3]:
```
{'orbit': 'Type (5)', 'normal_form': 'e123', 'real_rank': 1, 'alpha':
(1.00000000000000, 1.00000000000000, -0.000000000000000, 0.000000000000000,
-0.000000000000000, 0.000000000000000), 'beta': (0.000000000000000,
-0.000000000000000, -0.000000000000000, -0.000000000000000, -0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 1.00000000000000, 1.00000000000000, 1.00000000000000,
1.00000000000000, 0.000000000000000), 'beta_rank': 2}
============================================================
{'orbit': 'Type (4)', 'normal_form': 'e125 + e345', 'real_rank': 2, 'alpha':
(-0.000000000000000, -0.000000000000000, 0.000000000000000, -0.000000000000000,
1.00000000000000, -1.00000000000000), 'beta': (1.00000000000000, 0.000000000000000,
0.000000000000000, -0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, -0.000000000000000, 0.000000000000000, 1.00000000000000,
-0.000000000000000, 0.000000000000000, -0.000000000000000, 0.000000000000000,
0.000000000000000), 'beta_rank': 4}
============================================================
{'orbit': 'Type (6)', 'normal_form': '0', 'real_rank': 0, 'alpha':
(1.00000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta': (0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta_rank': 0}
============================================================
{'orbit': 'Type (4)', 'normal_form': 'e125 + e345', 'real_rank': 2, 'alpha':
(1.00000000000000, 1.00000000000000, 1.00000000000000, 1.00000000000000,
1.00000000000000, 1.00000000000000), 'beta': (1.00000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 1.00000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000), 'beta_rank': 4}
============================================================
{'orbit': 'Type (5)', 'normal_form': 'e123', 'real_rank': 1, 'alpha':
(1.00000000000000, 0.000000000000000, -0.000000000000000, -0.000000000000000,
-0.000000000000000, 0.000000000000000), 'beta': (0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
1.00000000000000, 1.00000000000000, 1.00000000000000, 1.00000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta_rank': 2}
============================================================
{'orbit': 'Type (6)', 'normal_form': '0', 'real_rank': 0, 'alpha':
(1.00000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta': (0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta_rank': 0}
============================================================
{'orbit': 'Type (4)', 'normal_form': 'e125 + e345', 'real_rank': 2, 'alpha':
(0.000000000000000, 0.000000000000000, 1.00000000000000, -0.000000000000000,
-0.000000000000000, 1.00000000000000), 'beta': (1.00000000000000,
0.000000000000000, -0.000000000000000, -0.000000000000000, 0.000000000000000,
```

```
0.000000000000000, -0.000000000000000, -0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, -1.00000000000000,
0.000000000000000, 0.000000000000000), 'beta_rank': 4}
============================================================
{'orbit': 'Type (5)', 'normal_form': 'e123', 'real_rank': 1, 'alpha':
(1.00000000000000, -0.000000000000000, -0.000000000000000, -0.000000000000000,
-0.000000000000000, -1.00000000000000), 'beta': (-0.000000000000000,
-0.000000000000000, -0.000000000000000, -0.000000000000000, 0.000000000000000,
1.00000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 1.00000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta_rank': 2}
============================================================
{'orbit': 'Type (4)', 'normal_form': 'e125 + e345', 'real_rank': 2, 'alpha':
(1.00000000000000, -0.000000000000000, -0.000000000000000, -0.000000000000000,
-0.000000000000000, -0.000000000000000), 'beta': (0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
2.00000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 2.00000000000000,
2.00000000000000, 0.000000000000000), 'beta_rank': 4}
============================================================
{'orbit': 'Type (6)', 'normal_form': '0', 'real_rank': 0, 'alpha':
(1.00000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta': (0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta_rank': 0}
============================================================
{'orbit': 'Type (5)', 'normal_form': 'e123', 'real_rank': 1, 'alpha':
(1.00000000000000, 0.000000000000000, 0.000000000000000, 1.00000000000000,
0.000000000000000, 0.000000000000000), 'beta': (1.00000000000000, 1.00000000000000,
0.000000000000000, -0.000000000000000, -0.000000000000000, 1.00000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000), 'beta_rank': 2}
============================================================
{'orbit': 'Type (4)', 'normal_form': 'e125 + e345', 'real_rank': 2, 'alpha':
(0.000000000000000, -0.000000000000000, -0.000000000000000, 1.00000000000000,
-0.000000000000000, -0.000000000000000), 'beta': (1.00000000000000,
0.000000000000000, 0.000000000000000, -0.000000000000000, -0.000000000000000,
0.000000000000000, 0.000000000000000, -0.000000000000000, 1.00000000000000,
0.000000000000000, -1.00000000000000, -0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta_rank': 4}
============================================================
{'orbit': 'Type (5)', 'normal_form': 'e123', 'real_rank': 1, 'alpha':
(1.00000000000000, -1.00000000000000, 0.000000000000000, -0.000000000000000,
0.000000000000000, -0.000000000000000), 'beta': (0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 1.00000000000000, -1.00000000000000, -1.00000000000000,
1.00000000000000, 0.000000000000000), 'beta_rank': 2}
============================================================
{'orbit': 'Type (4)', 'normal_form': 'e125 + e345', 'real_rank': 2, 'alpha':
(-0.000000000000000, 1.00000000000000, -0.000000000000000, -0.000000000000000,
-0.000000000000000, -0.000000000000000), 'beta': (0.000000000000000,
-0.000000000000000, -1.00000000000000, -0.000000000000000, -1.00000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
```

```
1.00000000000000, 0.000000000000000, 1.00000000000000, 1.00000000000000,
0.000000000000000, 1.00000000000000), 'beta_rank': 4}
============================================================
{'orbit': 'Type (6)', 'normal_form': '0', 'real_rank': 0, 'alpha':
(1.00000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta': (0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta_rank': 0}
============================================================
{'orbit': 'Type (5)', 'normal_form': 'e123', 'real_rank': 1, 'alpha':
(1.00000000000000, 1.00000000000000, 1.00000000000000, 0.000000000000000,
0.000000000000000, -0.000000000000000), 'beta': (0.000000000000000,
0.000000000000000, -0.000000000000000, -0.000000000000000, -0.000000000000000,
0.000000000000000, -0.000000000000000, -0.000000000000000, -0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 1.00000000000000,
0.000000000000000, 0.000000000000000), 'beta_rank': 2}
============================================================
{'orbit': 'Type (4)', 'normal_form': 'e125 + e345', 'real_rank': 2, 'alpha':
(1.00000000000000, 1.00000000000000, -0.000000000000000, -0.000000000000000,
-0.000000000000000, -0.000000000000000), 'beta': (0.000000000000000,
-0.000000000000000, -0.000000000000000, -0.000000000000000, -0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
1.00000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 1.00000000000000), 'beta_rank': 4}
============================================================
{'orbit': 'Type (6)', 'normal_form': '0', 'real_rank': 0, 'alpha':
(1.00000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta': (0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta_rank': 0}
============================================================
{'orbit': 'Type (5)', 'normal_form': 'e123', 'real_rank': 1, 'alpha':
(1.00000000000000, -0.000000000000000, -0.000000000000000, 1.00000000000000,
0.000000000000000, 0.000000000000000), 'beta': (0.000000000000000,
0.000000000000000, 0.000000000000000, -0.000000000000000, -0.000000000000000,
1.00000000000000, 0.000000000000000, 0.000000000000000, 1.00000000000000,
0.000000000000000, -1.00000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 1.00000000000000), 'beta_rank': 2}
============================================================
{'orbit': 'Type (4)', 'normal_form': 'e125 + e345', 'real_rank': 2, 'alpha':
(-0.000000000000000, -0.000000000000000, -0.000000000000000, -0.000000000000000,
-0.000000000000000, 1.00000000000000), 'beta': (1.00000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
3.00000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta_rank': 4}
============================================================
```

```
def random_3form(bound=5):

    alpha = {}

    for idx in basis3:
```

```
        c = randint(-bound, bound)

        if c != 0:
            alpha[idx] = RR(c)

    return alpha
def orbit_distribution(N=100):

    counts = {
        "Type (1)":0,
        "Type (2)":0,
        "Type (3)":0,
        "Type (4)":0,
        "Type (5)":0,
        "Type (6)":0,
        "Failed":0
    }

    for _ in range(N):

        alpha = random_3form()

        try:

            orbit = classify_full(alpha)["orbit"]

            counts[orbit] += 1

        except:

            counts["Failed"] += 1

    print("\nDistribution\n")

    for k,v in counts.items():

        pct = float(100*v/N)

        print(
            f"{k:10s} : {v:4d} ({pct:5.1f}%)"
        )
    print('''Type (1)
e1∧e2∧e3 + e4∧e5∧e6

Type (2)
e1∧e3∧e5 - e1∧e4∧e6 - e2∧e3∧e6 - e2∧e4∧e5

Type (3)
e1∧e5∧e6 + e2∧e6∧e4 + e3∧e4∧e5

Type (4)
e1∧e2∧e5 + e3∧e4∧e5

Type (5)
e1∧e2∧e3
```

```
Type (6)
0''')

    return counts
```

```
orbit_distribution(5000)
```

Out[4]:

```
Distribution

Type (1)   : 3409 ( 68.2%)
Type (2)   : 1588 ( 31.8%)
Type (3)   :    3 (  0.1%)
Type (4)   :    0 (  0.0%)
Type (5)   :    0 (  0.0%)
Type (6)   :    0 (  0.0%)
Failed     :    0 (  0.0%)
Type (1)
e1∧e2∧e3 + e4∧e5∧e6

Type (2)
e1∧e3∧e5 - e1∧e4∧e6 - e2∧e3∧e6 - e2∧e4∧e5

Type (3)
e1∧e5∧e6 + e2∧e6∧e4 + e3∧e4∧e5

Type (4)
e1∧e2∧e5 + e3∧e4∧e5

Type (5)
e1∧e2∧e3

Type (6)
0

{'Type (1)': 3409,
 'Type (2)': 1588,
 'Type (3)': 3,
 'Type (4)': 0,
 'Type (5)': 0,
 'Type (6)': 0,
 'Failed': 0}
```

```
test2 = [

    # 1. Negative Stable: Standard Type (2) canonical form
    {
        (0, 2, 4): 1, (0, 3, 5): -1, (1, 2, 5): -1, (1, 3, 4): -1
    },

    # 2. Positive Stable: Clean uncoupled blocks
    {
        (0, 1, 2): 1, (3, 4, 5): 1
    },

    # 3. Degenerate: Fully distributed Type (5) wedge product
```

```
{
    (0, 1, 2): 1, (0, 1, 5): 1, (0, 2, 4): -1, (0, 4, 5): 1,
    (1, 2, 3): 1, (1, 3, 5): -1, (2, 3, 4): 1, (3, 4, 5): 1
},

# 4. Negative Stable: Highly scrambled cross terms
{
    (0, 1, 4): 1, (0, 2, 3): -1, (1, 2, 5): 1, (3, 4, 5): 1
},

# 5. Degenerate: Type (4) layout sharing a common factor e5
{
    (0, 1, 5): 1, (2, 3, 5): 3
},

# 6. Positive Stable: Permuted index split
{
    (0, 3, 4): 1, (1, 2, 5): 1
},

# 7. Degenerate: Bryant Type (3) Boundary case (Indices internally sorted)
{
    (0, 4, 5): 1, (1, 3, 5): -1, (2, 3, 4): 1
},

# 8. Negative Stable: Asymmetric coefficients variant
{
    (0, 2, 4): 2, (0, 3, 5): -2, (1, 2, 5): -1, (1, 3, 4): -4
},

# 9. Positive Stable: Scrambled balanced block
{
    (0, 1, 3): 1, (2, 4, 5): 1
},

# 10. Degenerate: Standard Type (4) arrangement
{
    (0, 1, 4): 1, (2, 3, 4): 1
},

# 11. Negative Stable: Highly dense matrix coefficients
{
    (0, 1, 5): 1.5, (0, 2, 3): -1.5, (1, 2, 4): 1.5, (3, 4, 5): 1.5
},

# 12. Positive Stable: Symmetric block shift
{
    (0, 2, 5): 1, (1, 3, 4): 1
},

# 13. Degenerate: The trivial zero 3-form
{},

# 14. Negative Stable: Sign-inverted canonical variation
{
    (0, 2, 4): -1, (0, 3, 5): 1, (1, 2, 5): 1, (1, 3, 4): 1
},
```

```
    # 15. Positive Stable: Unique orientation pairing
    {
        (0, 1, 5): 1, (2, 3, 4): 1
    },

    # 16. Degenerate: Hyper-distributed Type (5) box case
    {
        (0, 2, 4): 1, (0, 2, 5): 1, (0, 3, 4): 1, (0, 3, 5): 1,
        (1, 2, 4): 1, (1, 2, 5): 1, (1, 3, 4): 1, (1, 3, 5): 1
    },

    # 17. Negative Stable: Skewed coefficient weightings
    {
        (0, 1, 4): 3, (0, 2, 3): -3, (1, 2, 5): 1, (3, 4, 5): 1
    },

    # 18. Positive Stable: Scale-imbalanced pairs
    {
        (0, 1, 2): 4, (3, 4, 5): 0.25
    },

    # 19. Degenerate: Shifted Type (4) configuration
    {
        (0, 1, 2): 1, (0, 1, 5): 1, (2, 3, 4): -1, (3, 4, 5): -1
    },

    # 20. Negative Stable: Shifted Type (2) canonical configuration
    {
        (0, 1, 2): 1, (0, 4, 5): -1, (1, 3, 5): 1, (2, 3, 4): -1
    },

    # 21. Positive Stable: Orthogonal structural split
    {
        (0, 1, 4): 1, (2, 3, 5): 1
    },

    # 22. Degenerate: Another clean empty/zero form checker
    {},

    # 23. Negative Stable: Large integer scale test
    {
        (0, 1, 2): 10, (0, 4, 5): -10, (1, 3, 5): 10, (2, 3, 4): -10
    },

    # 24. Positive Stable: Swapped component blocks
    {
        (0, 4, 5): 1, (1, 2, 3): 1
    },

    # 25. Degenerate: Coordinated boundary combinations
    {
        (0, 1, 3): 1, (0, 1, 4): 1, (2, 3, 5): 1, (2, 4, 5): 1
    }

]
for i in test2:
```

```
    print(classify_full(i))
    print("=="*50)
```

Out[54]:

```
{'orbit': 'Type (2)', 'lambda': -4.00000000000000, 'alpha': (0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.500000000000000, -0.500000000000000*I, -0.500000000000000*I, -0.500000000000000,
0.000000000000000, 0.000000000000000, -0.500000000000000*I, -0.500000000000000,
-0.500000000000000, 0.500000000000000*I, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000), 'alpha_bar':
(0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.500000000000000, 0.500000000000000*I, 0.500000000000000*I,
-0.500000000000000, 0.000000000000000, 0.000000000000000, 0.500000000000000*I,
-0.500000000000000, -0.500000000000000, -0.500000000000000*I, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000)}
==========================================================================================
{'orbit': 'Type (1)', 'lambda': 1.00000000000000, 'alpha': {(0, 1, 2):
1.00000000000000}, 'beta': {(3, 4, 5): 1.00000000000000}}
==========================================================================================
{'orbit': 'Type (5)', 'normal_form': 'e123', 'real_rank': 1, 'alpha':
(1.00000000000000, -0.000000000000000, -0.000000000000000, 1.00000000000000,
0.000000000000000, 0.000000000000000), 'beta': (0.000000000000000,
0.000000000000000, 0.000000000000000, -0.000000000000000, -0.000000000000000,
1.00000000000000, 0.000000000000000, 0.000000000000000, 1.00000000000000,
0.000000000000000, -1.00000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 1.00000000000000), 'beta_rank': 2}
==========================================================================================
{'orbit': 'Type (1)', 'lambda': 4.00000000000000, 'alpha': {(0, 1, 2):
0.500000000000000, (0, 1, 4): 0.500000000000000, (0, 2, 3): -0.500000000000000, (0,
3, 4): 0.500000000000000, (1, 2, 5): 0.500000000000000, (1, 4, 5):
0.500000000000000, (2, 3, 5): -0.500000000000000, (3, 4, 5): 0.500000000000000},
'beta': {(0, 1, 2): -0.500000000000000, (0, 1, 4): 0.500000000000000, (0, 2, 3):
-0.500000000000000, (0, 3, 4): -0.500000000000000, (1, 2, 5): 0.500000000000000,
(1, 4, 5): -0.500000000000000, (2, 3, 5): 0.500000000000000, (3, 4, 5):
0.500000000000000}}
==========================================================================================
{'orbit': 'Type (4)', 'normal_form': 'e125 + e345', 'real_rank': 2, 'alpha':
(-0.000000000000000, -0.000000000000000, -0.000000000000000, -0.000000000000000,
-0.000000000000000, 1.00000000000000), 'beta': (1.00000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
3.00000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta_rank': 4}
==========================================================================================
{'orbit': 'Type (1)', 'lambda': 1.00000000000000, 'alpha': {(0, 3, 4):
1.00000000000000}, 'beta': {(1, 2, 5): 1.00000000000000}}
==========================================================================================
{'orbit': 'Type (3)', 'lambda': 0}
==========================================================================================
{'orbit': 'Type (2)', 'lambda': -64.0000000000000, 'alpha': (0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
```

```
1.00000000000000, -0.500000000000000*I, -2.00000000000000*I, -1.00000000000000,
0.000000000000000, 0.000000000000000, -1.00000000000000*I, -0.500000000000000,
-2.00000000000000, 1.00000000000000*I, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000), 'alpha_bar':
(0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 1.00000000000000, 0.500000000000000*I, 2.00000000000000*I,
-1.00000000000000, 0.000000000000000, 0.000000000000000, 1.00000000000000*I,
-0.500000000000000, -2.00000000000000, -1.00000000000000*I, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000)}
================================================================================
=================
{'orbit': 'Type (1)', 'lambda': 1.00000000000000, 'alpha': {(2, 4, 5):
1.00000000000000}, 'beta': {(0, 1, 3): 1.00000000000000}}
================================================================================
=================
{'orbit': 'Type (4)', 'normal_form': 'e125 + e345', 'real_rank': 2, 'alpha':
(-0.000000000000000, -0.000000000000000, -0.000000000000000, -0.000000000000000,
1.00000000000000, -0.000000000000000), 'beta': (1.00000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, -0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, -0.000000000000000,
1.00000000000000, 0.000000000000000, -0.000000000000000, 0.000000000000000,
-0.000000000000000, 0.000000000000000), 'beta_rank': 4}
================================================================================
=================
{'orbit': 'Type (2)', 'lambda': -20.2500000000000, 'alpha': (0.750000000000000*I,
0.000000000000000, 0.000000000000000, 0.750000000000000, -0.750000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, -0.750000000000000*I,
0.000000000000000, 0.000000000000000, 0.750000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.750000000000000*I, 0.750000000000000*I,
0.000000000000000, 0.000000000000000, 0.750000000000000), 'alpha_bar':
(-0.750000000000000*I, 0.000000000000000, 0.000000000000000, 0.750000000000000,
-0.750000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.750000000000000*I, 0.000000000000000, 0.000000000000000, 0.750000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, -0.750000000000000*I,
-0.750000000000000*I, 0.000000000000000, 0.000000000000000, 0.750000000000000)}
================================================================================
=================
{'orbit': 'Type (1)', 'lambda': 1.00000000000000, 'alpha': {(0, 2, 5):
1.00000000000000}, 'beta': {(1, 3, 4): 1.00000000000000}}
================================================================================
=================
{'orbit': 'Type (6)', 'normal_form': '0', 'real_rank': 0, 'alpha':
(1.00000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta': (0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta_rank': 0}
================================================================================
=================
{'orbit': 'Type (2)', 'lambda': -4.00000000000000, 'alpha': (0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
-0.500000000000000, 0.500000000000000*I, 0.500000000000000*I, 0.500000000000000,
0.000000000000000, 0.000000000000000, 0.500000000000000*I, 0.500000000000000,
0.500000000000000, -0.500000000000000*I, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000), 'alpha_bar':
(0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
```

```
0.000000000000000, -0.500000000000000, -0.500000000000000*I, -0.500000000000000*I,
0.500000000000000, 0.000000000000000, 0.000000000000000, -0.500000000000000*I,
0.500000000000000, 0.500000000000000, 0.500000000000000*I, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000)}
================================================================================
=================
{'orbit': 'Type (1)', 'lambda': 1.00000000000000, 'alpha': {(2, 3, 4):
1.00000000000000}, 'beta': {(0, 1, 5): 1.00000000000000}}
================================================================================
=================
{'orbit': 'Type (5)', 'normal_form': 'e123', 'real_rank': 1, 'alpha':
(1.00000000000000, 1.00000000000000, -0.000000000000000, 0.000000000000000,
-0.000000000000000, 0.000000000000000), 'beta': (0.000000000000000,
-0.000000000000000, -0.000000000000000, -0.000000000000000, -0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 1.00000000000000, 1.00000000000000, 1.00000000000000,
1.00000000000000, 0.000000000000000), 'beta_rank': 2}
================================================================================
=================
{'orbit': 'Type (1)', 'lambda': 36.0000000000000, 'alpha': {(0, 1, 2):
1.50000000000000, (0, 1, 4): 1.50000000000000, (0, 2, 3): -1.50000000000000, (0, 3,
4): 1.50000000000000, (1, 2, 5): 0.500000000000000, (1, 4, 5): 0.500000000000000,
(2, 3, 5): -0.500000000000000, (3, 4, 5): 0.500000000000000}, 'beta': {(0, 1, 2):
-1.50000000000000, (0, 1, 4): 1.50000000000000, (0, 2, 3): -1.50000000000000, (0,
3, 4): -1.50000000000000, (1, 2, 5): 0.500000000000000, (1, 4, 5):
-0.500000000000000, (2, 3, 5): 0.500000000000000, (3, 4, 5): 0.500000000000000}}
================================================================================
=================
{'orbit': 'Type (1)', 'lambda': 1.00000000000000, 'alpha': {(0, 1, 2):
4.00000000000000}, 'beta': {(3, 4, 5): 0.250000000000000}}
================================================================================
=================
{'orbit': 'Type (4)', 'normal_form': 'e125 + e345', 'real_rank': 2, 'alpha':
(0.000000000000000, 0.000000000000000, 1.00000000000000, -0.000000000000000,
-0.000000000000000, 1.00000000000000), 'beta': (1.00000000000000,
0.000000000000000, -0.000000000000000, -0.000000000000000, 0.000000000000000,
0.000000000000000, -0.000000000000000, -0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, -1.00000000000000,
0.000000000000000, 0.000000000000000), 'beta_rank': 4}
================================================================================
=================
{'orbit': 'Type (2)', 'lambda': -4.00000000000000, 'alpha': (0.500000000000000,
0.000000000000000, 0.000000000000000, 0.500000000000000*I, 0.000000000000000,
-0.500000000000000*I, 0.000000000000000, 0.000000000000000, 0.000000000000000,
-0.500000000000000, 0.500000000000000*I, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.500000000000000, 0.000000000000000, -0.500000000000000,
0.000000000000000, 0.000000000000000, -0.500000000000000*I), 'alpha_bar':
(0.500000000000000, 0.000000000000000, 0.000000000000000, -0.500000000000000*I,
0.000000000000000, 0.500000000000000*I, 0.000000000000000, 0.000000000000000,
0.000000000000000, -0.500000000000000, -0.500000000000000*I, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.500000000000000, 0.000000000000000,
-0.500000000000000, 0.000000000000000, 0.000000000000000, 0.500000000000000*I)}
================================================================================
=================
{'orbit': 'Type (1)', 'lambda': 1.00000000000000, 'alpha': {(0, 1, 4):
1.00000000000000}, 'beta': {(2, 3, 5): 1.00000000000000}}
================================================================================
```

```
==================
{'orbit': 'Type (6)', 'normal_form': '0', 'real_rank': 0, 'alpha':
(1.00000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta': (0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, 0.000000000000000,
0.000000000000000, 0.000000000000000), 'beta_rank': 0}
================================================================================
==================
{'orbit': 'Type (2)', 'lambda': -40000.0000000000, 'alpha': (5.00000000000000,
0.000000000000000, 0.000000000000000, 5.00000000000000*I, 0.000000000000000,
-5.00000000000000*I, 0.000000000000000, 0.000000000000000, 0.000000000000000,
-5.00000000000000, 5.00000000000000*I, 0.000000000000000, 0.000000000000000,
0.000000000000000, 5.00000000000000, 0.000000000000000, -5.00000000000000,
0.000000000000000, 0.000000000000000, -5.00000000000000*I), 'alpha_bar':
(5.00000000000000, 0.000000000000000, 0.000000000000000, -5.00000000000000*I,
0.000000000000000, 5.00000000000000*I, 0.000000000000000, 0.000000000000000,
0.000000000000000, -5.00000000000000, -5.00000000000000*I, 0.000000000000000,
0.000000000000000, 0.000000000000000, 5.00000000000000, 0.000000000000000,
-5.00000000000000, 0.000000000000000, 0.000000000000000, 5.00000000000000*I)}
================================================================================
==================
{'orbit': 'Type (1)', 'lambda': 1.00000000000000, 'alpha': {(0, 4, 5):
1.00000000000000}, 'beta': {(1, 2, 3): 1.00000000000000}}
================================================================================
==================
{'orbit': 'Type (4)', 'normal_form': 'e125 + e345', 'real_rank': 2, 'alpha':
(-0.000000000000000, -0.000000000000000, -0.000000000000000, 1.00000000000000,
1.00000000000000, -0.000000000000000), 'beta': (1.00000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, -0.000000000000000,
0.000000000000000, 0.000000000000000, 0.000000000000000, -0.000000000000000,
0.000000000000000, 0.000000000000000, -1.00000000000000, 0.000000000000000,
-0.000000000000000, 0.000000000000000), 'beta_rank': 4}
================================================================================
==================
```